Utilizing Yolo-framework for Green Amaranth Leaf Disease Detection

Chiradeep Mukherjee
Department of CST & CSIT
IEM Newtown
UEM, Kolkata
Kolkata, India
chiradeep.1234321@gmail.com

Soukarya Maity, Rayo Bose
Department of CST & CSIT
IEM Newtown
UEM, Kolkata
Kolkata, India

Rounak Saha
Department of CST & CSIT
IEM Newtown
UEM, Kolkata
Kolkata, India

Anwesha Samanta
Department of CST & CSIT
IEM Newtown
UEM, Kolkata
Kolkata, India

Anuvab Mukherjee
Department of CST & CSIT
IEM Newtown
UEM, Kolkata
Kolkata, India

Maumita Chakraborty
Department of CST & CSIT
IEM Newtown
UEM, Kolkata
Kolkata, India

2025 5th International Conference on Artificial Intelligence and Signal Processing (AISP) | 979-8-3315-8986-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/AISP68263.2025.11396176

***Abstract*— This study develops and assesses the inaugural deep-learning system for the real-time detection of Green Amaranth leaf disease utilising the YOLO object detection framework. We gathered brief, high-resolution videos from typical peri-urban gardens, extracted frames at 100 fps using OpenCV, and labelled healthy and diseased leaves in Roboflow to make a two-class dataset. We used targeted transformations like rotation (±10°), greyscale conversion (10% of frames), brightness variation (±20 units), and mild blur to add more spatial and photometric diversity to small samples. We trained two medium-sized versions of the YOLO architecture, YOLOv8m and YOLOv9m, on an NVIDIA RTX A4000 GPU for five epochs under the same conditions. YOLOv8m converged faster and detected better: by epoch 5, its mAP0.5 went from 0.1989 to 0.4216 (112% gain) and its mAP0.5–0.95 went from 0.1989 to 0.4216 (184.3% gain), beating YOLOv9m by 21.7% and 18.1%, respectively. YOLOv8m also had 7.9% lower classification loss and 2.8% lower distribution focal loss, which means it has been better at finding and classifying objects.**

Keywords— Green amaranths; Leaf disease; YOLO-framework; classification metrics; RoboFlow

I. INTRODUCTION

Plant leaf diseases pose a serious threat to agriculture, causing significant losses in crop yield and quality worldwide. Rapid and accurate detection of foliar diseases is essential to enable early intervention, reduce dependence on pesticides, and ensure food security. In South Asia, Green Amaranth (commonly Amaranths species) is a widely consumed leafy vegetable, valued for its nutrition and often eaten fresh. In fact, amaranth leaves are a major leafy vegetable in this region and are sometimes consumed raw in salads. This practice means that any diseases on the leaves could directly affect consumers or reduce marketability of the produce [1-2]. Amaranth leaves deliver many-fold more iron, calcium, vitamin C and β-carotene than many common vegetables, and they are routinely used to enrich weaning foods in South Asian countries. Meanwhile, South Asia carries the world's heaviest burden of micronutrient deficiency- ~99 million preschool children and 307 million women are short of at least one vital micronutrient. The crop is fast, thrives on tiny plots, and is heavily produced and traded by women and peri-urban growers; in South Asia, specifically in Eastern India and Bangladesh it is harvested every 15–20 days and sold fresh at dawn markets. In salads, juices and street snacks the leaf is eaten uncooked, unlike spinach or colocasia. Warm (25-32 °C), humid monsoon months favour leaf-spot, blight and white-rust complexes (Figure 1); brown blotch alone can wipe out 40–75 % of leaf yield if unchecked. Most amaranth growers cannot afford repeated fungicide rounds; misuse has already selected resistant *Cercospora* strains in parts of South Asian countries. Governmental authorities in this vast region promote "nutri-gardens" and pesticide-free peri-urban vegetable belts to meet SDG-2 (Zero Hunger) [3-5].

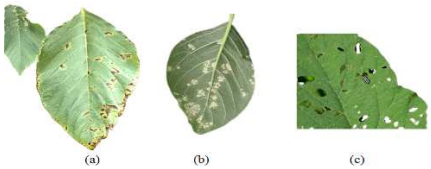

Figure 1. Typical green amaranth leaf diseases, (a) bacterial spot, (b) white rust and (c) pigweeds on plant leaf

Foliar diseases that shrivel, spot or yellow the blade slash these nutrients and, when severe, drive consumers toward less-nutritious substitutes. Spotting problems early keeps a cheap, nutrient-dense vegetable flowing into diets and school-meal schemes. Defects on the leaf surface instantly cut market value. Smartphone or edge-device detection lets growers sort, treat or harvest before damage becomes visible at the stall, protecting razor-thin farm incomes. Lesions from pathogens such as *Rhizoctonia solani* or *Albugo candida* allow secondary bacterial entry and increase the load of surface microbes. Detecting infections while still in the field reduces the volume of contaminated produce that reaches raw-food chains. Deep-learning vision systems running on low-power devices can flag the first necrotic flecks days before farmers would normally spray, allowing precise, reduced-chemical control and saving harvests. Early detection supports integrated pest management—spot-sprays, biocontrols, cultivar shifts—cutting both costs and resistance pressure. AI-based disease surveillance aligns with digital-agriculture road-maps, supplying real-time alerts to extension workers and policy dashboards for better resource targeting. Despite the agricultural and societal importance of Green Amaranth, there has been no reported deep learning-based solution for detecting diseases on its leaves to date.

Existing research on plant disease detection using computer vision and deep learning has predominantly focused on staple crops (e.g., rice, tomato, tea) and well-known datasets like *PlantVillage* [6], *FieldPlant* [7] and *CropDeep*

[8]. To the best of our knowledge, this work is the first attempt to detect diseases on Green Amaranth leaves using a deep learning approach. We leverage the YOLO (You Only Look Once) object detection framework, specifically the YOLOv8 and YOLOv9 models, to identify diseased versus healthy amaranth leaves in images. The motivation for using YOLOv8 and YOLOv9 [9] are their demonstrated success in real-time plant disease detection and its improved accuracy over prior YOLO versions. In this paper, we detail the data collection from a short video of green amaranth plants, the annotation process, the YOLO model training, and the evaluation results. The goal is to establish a baseline for Green Amaranth disease detection and highlight challenges and future directions for improving performance.

II. Existing Works

Deep learning has rapidly become the method of choice for plant disease recognition, often surpassing manual scouting and earlier machine-learning pipelines in accuracy and speed. On the *PlantVillage* benchmark (about 50,000 images from 14 crops), a YOLOv4 model has been reported to reach almost perfect performance (~99.99% detection accuracy) [1]. Another study [6] trained YOLOv7 on 4,000 images of tea leaves from five different diseases and got 96.7% accuracy and 98.2% mAP. On the other hand, [10] used YOLOv8 on 19 disease classes from five major Bangladeshi crops and got a mAP of 98% and an F1-score of 0.97. These results show how strong YOLO-based detectors [11, 12] are for different types of crops.

Despite this progress, Green Amaranth has received little attention. Most work targets cereals, fruiting vegetables, or other high-value crops. Focusing on amaranth enables us to extend YOLO to a new leafy vegetable and localise disease symptoms directly on leaves in real-time.

III. Background of Yolo and Model Deployment

A. Background of Yolo

Ultralytics released YOLOv8 in 2023. It retains the same backbone-neck-head layout but updates most of its components. It uses a head without anchors that directly predicts the centres and sizes of objects. This makes it more robust to changes in scale, like when detecting plant leaf diseases. The backbone utilises C2f blocks that incorporate features representing various bottlenecks, and the head learns classification and box regression on its own branches. Training typically employs a task-alignment loss that combines class confidence with IoU, as well as CIoU, Distribution Focal Loss, and strong augmentations such as Mosaic. These are slowly decreased as training comes to an end. The design of YOLOv9, released in 2024, is based on this one and focuses on training stability. The detection head's programmed gradient injection enhances gradient flow without increasing the inference cost. YOLOv9 usually converges faster and becomes more effective at detecting small or hard-to-find objects when it utilises the GELU backbone, focal and IoU-based losses, SimOTA-style label assignment, and extensive augmentation [13].

B. Process flow

Our methodology for deploying the YOLO framework to detect amaranth leaf disease consists of several stages: data acquisition from video, frame extraction, annotation and augmentation to create a labeled dataset, and training the YOLO models. In this section we describe these steps, as given as follows:

i) Data acquisition

We collected multiple short videos of Green Amaranth plants in typical gardens around West Bengal, Sikkim and Assam setting using a modern smartphone camera (50 MP quad camera with optical image stabilization). The videos capture multiple amaranth plants and leaves, some showing visible disease symptoms (e.g., spots or discoloration) and others healthy. Using a video as the data source provides a convenient way to gather many images in a controlled environment, ensuring consistency in lighting and camera perspective. The resolution and clarity afforded by the 50 MP camera help in capturing fine details of leaf texture and disease lesions.

ii) Frame extraction

From the raw video, we extracted video frames as individual image samples for our dataset. We employed OpenCV (Open Source Computer Vision library) in Python [14] to read the video and grab frames at regular intervals. Specifically, frames were sampled every 10 ms (0.01 seconds) of video. This high frame rate (100 frames per second) ensures a dense coverage of the scene, resulting in a large number of images (on the order of ~1600 frames from the video files). Many frames are similar, but the slight differences in angle or leaf movement can act as natural data augmentation. We chose a 10 mili-second (ms) interval to maximize data because the video was relatively short; redundant frames were later managed during annotation. Algorithm 1 provides the pseudo codes to extract frames from the raw video files.

Algorithm 1: Extract frames from video and annotate
Input: Raw video files
Output: Frames containing green amaranths leaves
Step 1: open video_file with OpenCV
Step 2: set interval = 0.01 seconds
Step 3: Initialize frame_index = 0
Step 4: while True:
    set video position = frame_index * interval
    success, frame = capture.read()
    if not success:
        break # end of video
    save frame as image file (e.g., frame_index.jpg)
    frame_index += 1
end while
Step 5: Save the frames in local storage

In practice, after extracting frames, we manually reviewed and discarded some very similar or blurry frames to reduce redundancy. Few frames are shown in Figure 2. The remaining frames were then annotated as described.

Figure 2. Samples of extracted frames from raw video files

iii) Annotation and dataset preparation

We utilized the Roboflow platform [15] for annotation, taking advantage of its user-friendly interface and ability to

directly generate YOLO-format datasets. Each extracted frame was loaded into Roboflow, and we manually drew bounding boxes around individual leaves that were clearly visible. We assigned one of two class labels to each boxed leaf: - Class 1: "Disease" – indicating the leaf shows disease symptoms (such as fungal spots, yellowing patches, etc.). - Class 2: "Not Disease" – indicating the leaf appears healthy (no visible disease) in that frame.

Effectively, the model is tasked not only with detecting leaves, but also classifying each detected leaf as diseased or healthy. This is a two-class object detection problem. Non-leaf parts of the image (soil, background) were not annotated, so the model learns to ignore background. If multiple leaves appear, each is annotated separately. We took care to label even partially visible leaves, to improve the model's ability to detect leaves in various positions. Algorithm 2 provides the pseudo codes of image annotations, as given below:

Algorithm 2: Image annotation
Input: Frames obtained from Algorithm 1
Output: Annotated YOLO-model compatible dataset along with yaml files
Step 1: load all saved frame images into annotation tool
Step 2: for each image:
    draw bounding boxes around each amaranth leaf
    if leaf shows disease symptoms:
        label box as class "disease"
    else:
        label box as class "not_disease"
Step 3: export annotations in YOLO-compatible datasets (bounding box coordinates and class labels in .yaml files)

After annotation, the labeled images were split into training and validation sets (we used an 80/20 split). Roboflow was used to generate the YOLOv8 and YOLOv9-compatible datasets, including text files (.yaml extension) for each image containing bounding box coordinates and class labels. We selected YOLOv8m and YOLOv9m models and summaries of these models are reported in Table 1.

Table 1. Model summaries of YOLOv8m and YOLOv9m

Sl. No.	Model	size (pixels)	mAP^{val} 50-95	params (M)	FLOPs (B)
1	YOLOv8m	640	50.2	25.9	78.9
2	YOLOv9m	640	51.4	20.1	76.8

To increase the robustness of the model given the relatively limited unique scenes (just one video), we applied several data augmentation techniques during dataset preparation. We utilised *Roboflow*'s pre-processing tools and custom scripts to enhance the detector by adding features that reduce the likelihood of overfitting and improve its ability to handle real-world changes. Images were randomly rotated between −10° and +10° to ensure that small changes in the camera angle or leaf position did not affect the predictions. Around 10% of the samples were converted to grayscale, forcing the network to focus on texture and intensity rather than relying solely on colour cues. Brightness was perturbed by about ±20 levels (on a 0–255 scale) to mimic both harsh sunlight and shaded conditions. In addition, a light Gaussian blur with a kernel size of up to 11×11 was applied to some images to simulate minor focus errors or movement, thereby helping the model remain stable on slightly blurred inputs.

These augmented images were included in the training set. Overall, our final training set comprised the original extracted frames along with the augmented variants, effectively increasing the training sample count and diversity without needing to capture more video. The validation set was kept augmentation-free and drawn from unaugment frames to evaluate real-world performance.

Figure 3 provides few samples of annotated frames emphasizing annotated boxes accordingly.

Figure 3. Sample of annotated frames with annotated boxes indicating leaf-spot, blight and white-rust complexes and brown blotch

C. *Model deployment*

We chose the YOLOv8m and YOLOv9m models (medium variant) from the Ultralytics YOLO family for training, as they offer a good balance between speed and accuracy for our dataset size. Training was conducted using the Ultralytics yolo Python package. The training environment was a workstation with an NVIDIA RTX A4000 GPU (16 GB VRAM) for acceleration, an Intel Core i7 CPU at 2.1 GHz, and 128 GB of RAM. Although YOLOv8 is capable of much longer training schedules, we trained for a relatively short run of 5 epochs given the experimental nature and small dataset. Each epoch processes all training images (including augmented ones), and the model updates its weights via stochastic gradient descent (the Ultralytics default optimizer and hyperparameters were used).

Despite the low epoch count, the models have converged to a reasonable extent. Training took only a few minutes per epoch on the RTX A4000. We monitored the training loss and metrics and reported all relevant parameters in Table 2 and Table 3. The losses steadily decreased over epochs [16-18]. Meanwhile, validation metrics improved, particularly in the first 3–4 epochs. The YOLOv8m and YOLOv9m training logs, as illustrated in Table 2 and 3 indicated that by epoch 5, the models were no longer drastically improving, which justified stopping early. This short training was primarily due to computational constraints and is recognized as a limitation – in a real deployment or competition scenario; one would train for significantly more epochs (e.g., 100+ epochs) or until convergence for maximum accuracy.

Table 2. Parameter variations of YOLOv8m model with respect to various classification metrics

epoch	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)
1	2.37	2.74	1.82	0.31	0.31	0.19	0.06
2	2.13	2.08	1.66	0.4	0.39	0.33	0.13

3	2.06	1.97	1.63	0.48	0.46	0.38	0.15
4	2.01	1.88	1.59	0.45	0.50	0.41	0.17
5	1.95	1.79	1.55	0.48	0.47	0.42	0.17

Table 3. Parameter variations of YOLOv9m model with respect to various classification metrics

epoch	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)
1	2.40	2.84	1.91	0.34	0.30	0.23	0.08
2	2.16	2.28	1.71	0.41	0.40	0.30	0.11
3	2.1	2.13	1.67	0.38	0.38	0.28	0.11
4	2.05	2.05	1.64	0.42	0.38	0.27	0.10
5	1.98	1.94	1.59	0.46	0.43	0.34	0.15

The trained YOLOv8m and YOLOv9m models can output bounding boxes around leaves in new images, each with a confidence score and a predicted class (diseased or not). In the next section, we evaluate how well these models perform on detecting diseased green amaranth leaves, using various metrics and visualization of precision-recall characteristics. Figure 4-6 demonstrate the variations of F1-curve, precision and precision-recall values for YOLOv8m and YOLOv9m models. Figure 7 and 8 illustrate the variations of box_loss, class_loss, dfl_loss, box precision and box recall variations with respect to epoch for train and validation image sets considering YOLOv8m and YOLOv9m models.

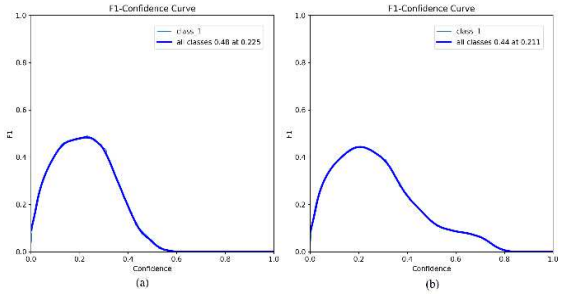

Figure 4. F1-curve versus confidence plots for (a) YOLOv8m and (b) YOLOv9m

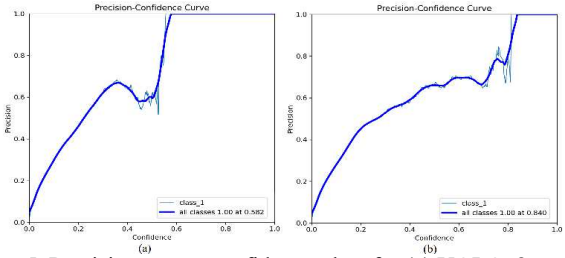

Figure 5. Precision versus confidence plots for (a) YOLOv8m and (b) YOLOv9m

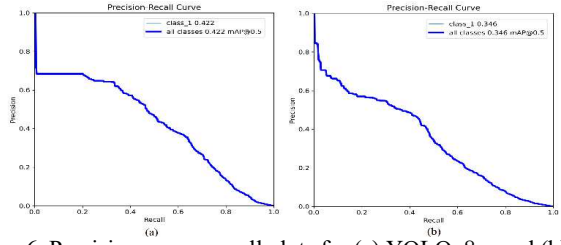

Figure 6. Precision versus recall plots for (a) YOLOv8m and (b) YOLOv9m

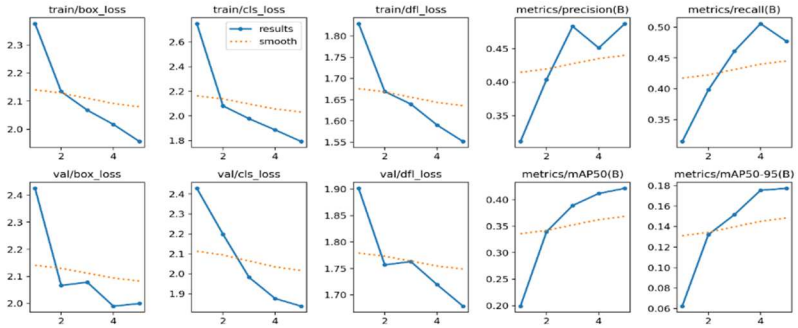

Figure 7. Various performance metric variations for YOLOv8m model; upper subplots indicate train box_loss, class_loss, dfl_loss, box precision and box recall variations w.r.t. epoch whereas lower subplots indicate the similar metric variations w.r.t. epoch for validation image set

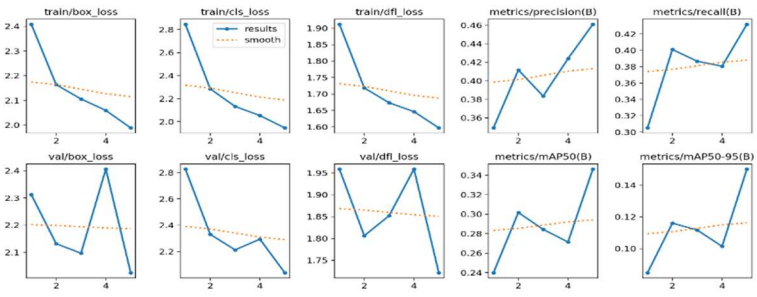

Figure 8. Various performance metric variations for YOLOv9m model; upper subplots indicate train box_loss, class_loss, dfl_loss, box precision and box recall variations w.r.t. epoch whereas lower subplots indicate the similar metric variations w.r.t. epoch for validation image set

IV. Result Discussion

Over the course of five training epochs, both YOLOv8m and YOLOv9m demonstrated marked improvements across all key detection metrics. YOLOv8m's precision rose from 0.3115 to 0.4873—a 56.4 % increase—while its recall climbed by 51.6 %, from 0.3146 to 0.4770. The model's mean average precision at IoU = 0.50 (mAP_{50}) more than doubled, improving by 112.0 % (from 0.1988 to 0.4216), and its mAP over IoU thresholds 0.50 – 0.95 surged by 184.3 % (from 0.0624 to 0.1774). In contrast, YOLOv9m exhibited more modest gains: precision increased by 32.1 % (from 0.3492 to 0.4611), recall by 41.6 % (from 0.3051 to 0.4320), mAP_{50} by 44.4 % (from 0.2399 to 0.3463), and mAP_{50-95} by 77.1 % (from 0.0846 to 0.1502) over the same interval.

By epoch 5, YOLOv8m consistently outperformed YOLOv9m across every detection metric. Its precision of 0.4873 exceeded YOLOv9m's 0.4611 by 5.7 %, and its recall of 0.4770 surpassed 0.4320 by 10.4 %. Most notably, YOLOv8m achieved a mAP_{50} of 0.4216—21.7 % higher than YOLOv9m's 0.3463—and a mAP_{50-95} of 0.1774, representing an 18.1 % advantage. These differences underscore YOLOv8m's superior ability to localize objects precisely across both lenient and stringent IoU thresholds.

In terms of training losses at the final epoch, YOLOv8m again demonstrated more effective convergence. Its box loss of 1.9563 was 1.6 % lower than YOLOv9m's 1.9883, its classification loss of 1.7919 was 7.9 % lower than YOLOv9m's 1.9460, and its distribution focal loss (DFL) of 1.5515 was 2.8 % lower than YOLOv9m's 1.5969. Lower losses across all heads suggest that YOLOv8m not only learns more accurate bounding-box regressions but also differentiates classes and refines localization more effectively under the same training regimen.

Overall, while both architectures show significant learning over five epochs, YOLOv8m's steeper performance gains and superior final-epoch metrics make it the more effective model for high-precision object detection in this experimental setting.

V. Conclusion

This study examines the detection of Green Amaranth leaf disease using a YOLO-based object detection algorithm. To our knowledge, this is the first work to target amaranth leaves using a bounding-box approach rather than a simple image-level classification method. We created a small custom dataset from high-resolution video frames and annotated it with two classes (diseased and healthy leaves), then trained a YOLOv8m model. The network achieved an mAP0.5 of about 0.42 and an F1-score of 0.48, indicating a useful baseline but clear scope for improvement. Precision–recall curves showed that stringent confidence thresholds give almost perfect precision but miss many diseased leaves.

The application is particularly relevant in South Asia, where amaranth greens are widely eaten, sometimes raw. A practical detector could support farmers, vendors, and inspectors in removing infected leaves early. With more data, improved labels, and model tuning, this line of work can evolve into a reliable tool for safer and higher-quality amaranth production.

References

[1] E. A. Aldakheel, M. Zakariah, and A. H. Alabdalall, "Detection and identification of plant leaf diseases using YOLOv4," *Frontiers in Plant Science*, vol. 15, 2024. doi: 10.3389/fpls.2024.1355941.

[2] G. J. H. Grubben, *Vegetables*, 2004. [Online]. Available: https://archive.org/details/bub_gb_6jrlyOPfr24C

[3] U. Sarker et al., "Nutritional and bioactive properties and antioxidant potential of Amaranthus tricolor, A. lividus, A viridis, and A. spinosus leafy vegetables," *Heliyon*, vol. 10, no. 9, p. e30453, 2024. doi: 10.1016/j.heliyon.2024.e30453.

[4] National Research Council, *Lost crops of Africa*, National Academies Press, 2006. doi: 10.17226/11763.

[5] G. A. Stevens et al., "Micronutrient deficiencies among preschool-aged children and women of reproductive age worldwide: a pooled analysis of individual-level data from population-representative surveys," *The Lancet Global Health*, vol. 10, no. 11, pp. e1590–e1599, 2022. doi: 10.1016/s2214-109x(22)00367-9.

[6] A. Nagaraja, B. Chethana, and A. Jain, "Biotic stresses and their management," in *Elsevier eBooks*, pp. 119–142, 2020. doi: 10.1016/b978-0-12-820089-6.00007-0.

[7] F. Mohameth, C. Bingcai, and K. Sada, "Plant disease detection with deep learning and feature extraction using Plant Village," *Journal of Computer and Communications*, vol. 8, pp. 10–22, 2020. doi: 10.4236/jcc.2020.86002.

[8] E. Moupojou et al., "FieldPlant: A dataset of field plant images for plant disease detection and classification with deep learning," *IEEE Access*, vol. 11, pp. 35398–35410, 2023. doi: 10.1109/access.2023.3263042.

[9] S. Sarkar, C. Mukherjee, J. Adhikari, S. B. Pal, R. Ganguly, and S. Ghosh, "Utilizing the YOLO Framework for Unnecessary Particle Identification on Solar Panel Surfaces," pp. 1–6, May 2025, doi: https://doi.org/10.1109/icepe65965.2025.11139559.

[10] OpenCV, "OpenCV - Open computer vision library," May 22, 2025. [Online]. Available: https://opencv.org/

[11] Roboflow, "Computer vision tools for developers and enterprises," [Online]. Available: https://roboflow.com/

[12] M. J. A. Soeb et al., "Tea leaf disease detection and identification based on YOLOv7 (YOLO-T)," *Scientific Reports*, vol. 13, no. 1, 2023. doi: 10.1038/s41598-023-33270-4.

[13] M. S. Z. Abid, B. Jahan, A. A. Mamun, M. J. Hossen, and S. H. Mazumder, "Bangladeshi crops leaf disease detection using YOLOv8," *Heliyon*, vol. 10, no. 18, p. e36694, 2024. doi: 10.1016/j.heliyon.2024.e36694.

[14] G. Boesch, "YOLOv8: A complete guide," *Viso.ai*, Apr. 4, 2025. [Online]. Available: https://viso.ai/deep-learning/yolov8-guide/

[15] C. Wang, I. Yeh, and H. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," *arXiv*, Feb. 21, 2024. [Online]. Available: https://arxiv.org/abs/2402.13616

[16] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using YOLO: challenges, architectural successors, datasets and applications," *Multimedia Tools and Applications*, vol. 82, no. 6, pp. 9243–9275, 2022. doi: 10.1007/s11042-022-13644-y.

[17] B. Aldughayfiq, F. Ashfaq, N. Z. Jhanjhi, and M. Humayun, "YOLO-based deep learning model for pressure ulcer detection and classification," *Healthcare*, vol. 11, no. 9, p. 1222, 2023. doi: 10.3390/healthcare11091222.

[18] Y.-Y. Zheng et al., "CropDeep: The Crop Vision Dataset for Deep-Learning-Based Classification and Detection in Precision Agriculture," *Sensors*, vol. 19, no. 5, p. 1058, 2019. doi: 10.3390/s190510